Mastering Uncertainty

Awareness, Reduction and Management

Follow along at:
https://reliableai.github.io/ => Playground

“Any measurement, without knowledge of the uncertainty, is meaningless.”

- Walter Lewin

“I WANT YOU TO HEAR THIS AT 3AM TONIGHT WHEN YOU WAKE UP”

"Any measurement, without knowledge of the uncertainty, is meaningless."

- Walter Lewin

*"If such measurement is reported, it goes from meaningless to **misleading**"*

- Just me

How “reliable” is this measure (and its color)?

Technical accuracy

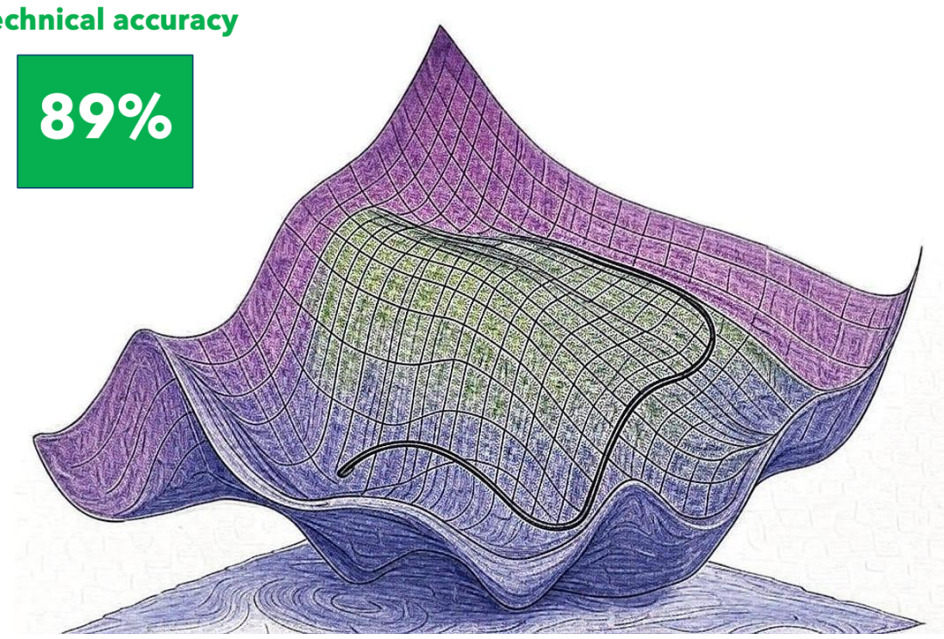
89%

“Eval” in AI systems has a foundational role.

1. Ship / no ship decision
2. They tell us where to improve

SW companies are awesome once they know what to optimize for.

With accurate “eval” you *will* get a good system



Should I Care About Uncertainty?

01 Which kinds of Uncertainty can there be?

02 Where does it come from?

03 Does it matter in practice?

04 What can we do about it?

Prelude: **Bias**, **Noise** and **Variance** as used in this presentation

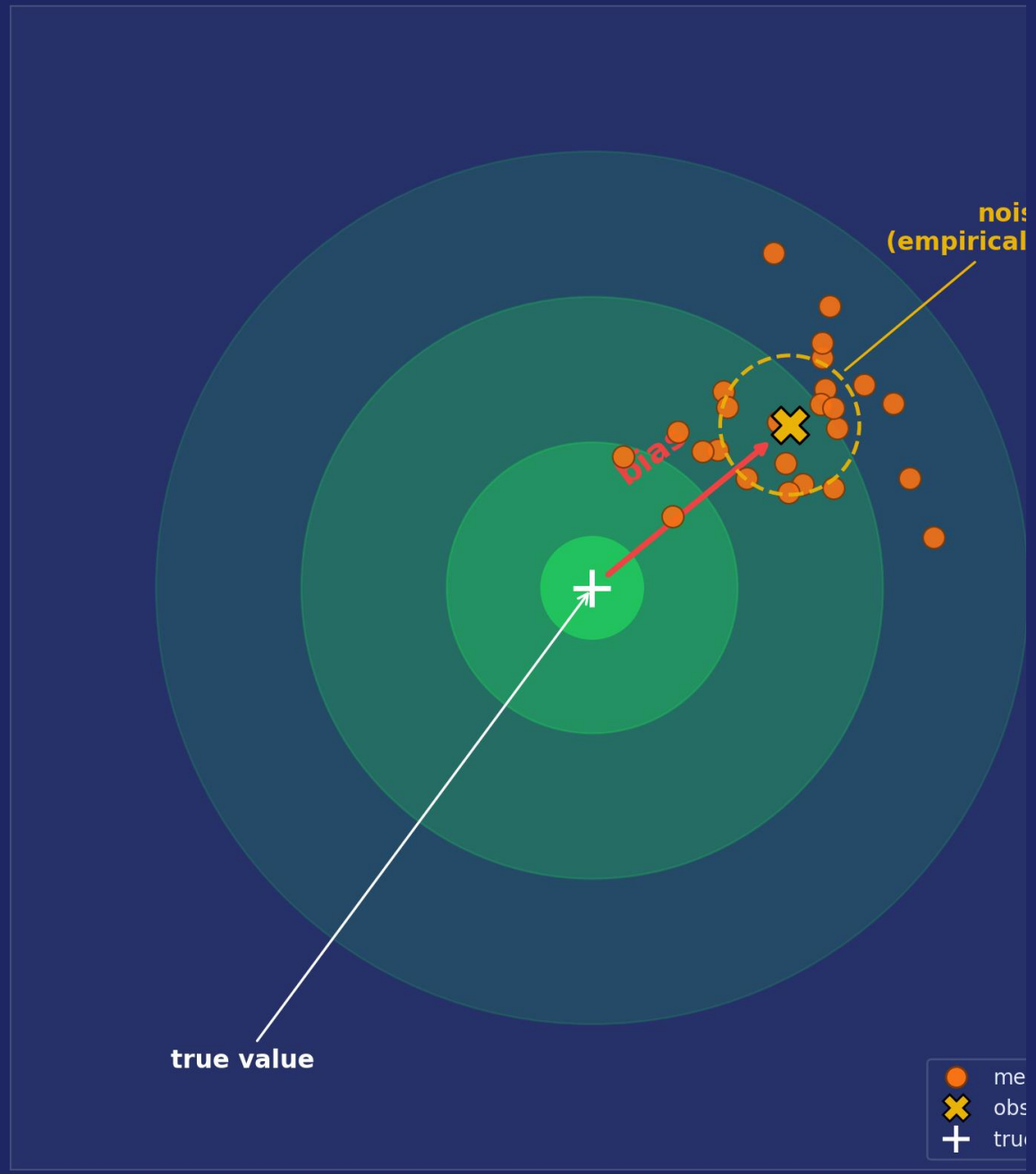
The official definition of these terms doesn't matter. It matters what people understand when you say these words. In our context:

Bias: a deviation of our estimate from reality due to a process that is likely to lead to overestimating (or underestimating) the quantity of interest. An example is a process that leads us to overestimate accuracy.

Noise: a random error in the estimation. For example, we take a sample, and that sample will have a mean that is different from the population mean due to small sample size. More data makes our estimate more precise and reduce the effect of noise.

Transfer Variance: We will often use this term to refer to variance across populations or time – and so, for example, variability due to different customers having different data characteristics. This cannot be reduced with more data.

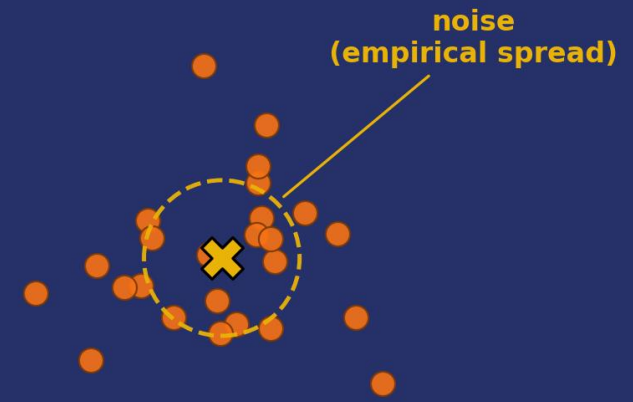
With ground truth — bias AND noise visible

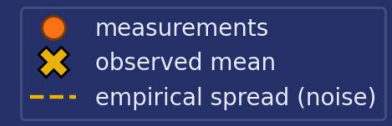


Without ground truth — only noise is measurable

We can still compute:
- observed mean
- empirical std / CI
- bootstrap distribution

We CANNOT compute:
- bias
- distance from truth
- whether our answer is correct





Example: MSE

Mean Squared Error (MSE)

$$E[(\hat{Y} - Y)^2]$$

$\hat{Y}$ = our prediction (estimate)

Y = true value

Only Bias

Model

$$\hat{Y} = Y + b$$

Error

$$E[(\hat{Y} - Y)^2] = b^2$$

Error = Bias²

The prediction is systematically shifted from the true value by a fixed amount b.

STEP 2

Bias + Noise

Model

$$\hat{Y} = Y + b + \varepsilon$$

where $E[\varepsilon] = 0$, $Var(\varepsilon) = \sigma^2$

Error

$$E[(\hat{Y} - Y)^2] = b^2 + \sigma^2$$

Error = Bias² + Noise

Both bias AND noise contribute to the error in MSE

How We Measure Error

$$
\begin{aligned}
&E[(\theta_{est} - \theta_{real})^2] \;\; = \\
&\quad (\text{bias from process})^2 \\
&+ \; (\text{sampling noise in } \theta_{est}) \\
&+ \; (\theta_{train} - \theta_{deploy})^2
\end{aligned}
$$

This is the “transfer variance”

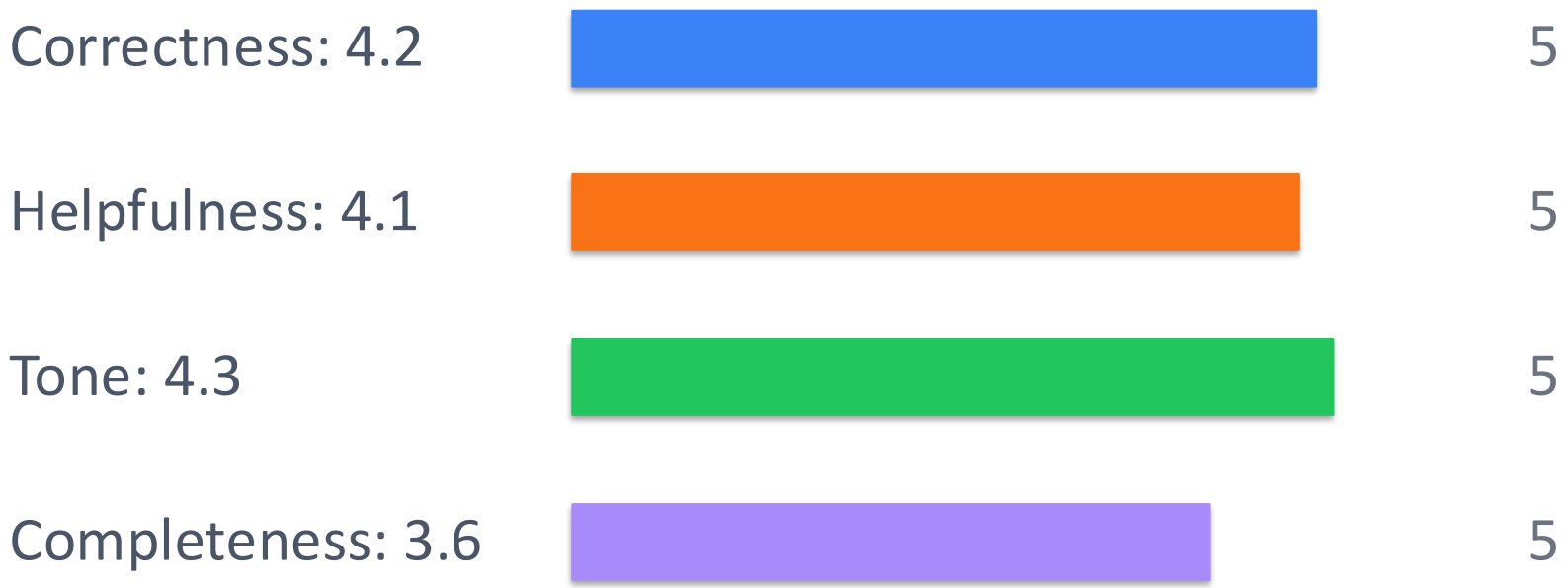
02

Types and Sources of Uncertainty

Understanding where and why evaluations break down

The Setup

AI System → LLM Judge → 1-5 Scores
4-Metric Scorecard: Correctness, Helpfulness, Tone, Completeness



Sources of Uncertainty + One Cross-cutting Layer

01 Sampling Noise

02 Sampling Bias

03 Overfitting

04 Multiple Hypotheses

05 Domain Variance

06 Choice of Metric

07 Temporal Drift

08 Brittleness

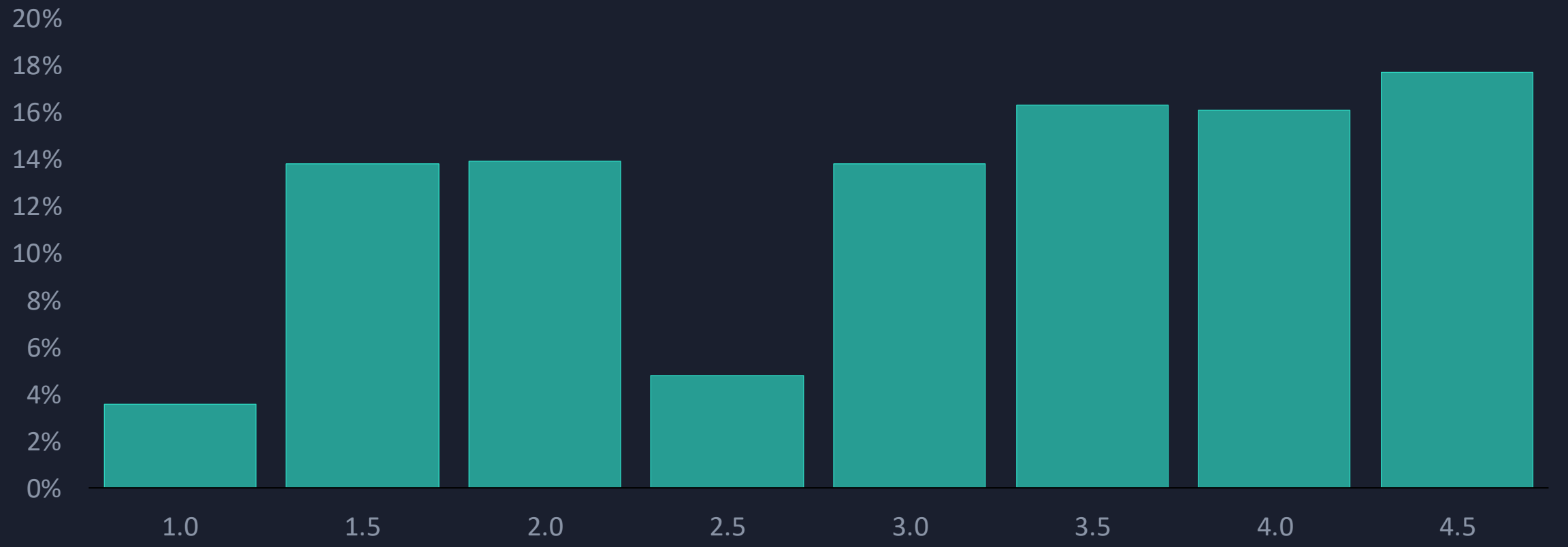
The Judge Layer — Cross-cutting uncertainty layer

01

Sampling Noise

This is what people think you mean when you bring up uncertainty

Our Unknown True Distribution

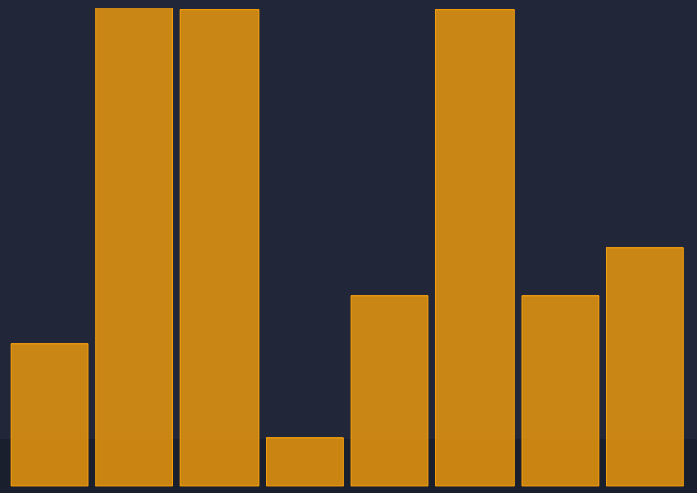


Population: **Mean = 3.32** | **Median = 3.50** | **%>4 = 34%**

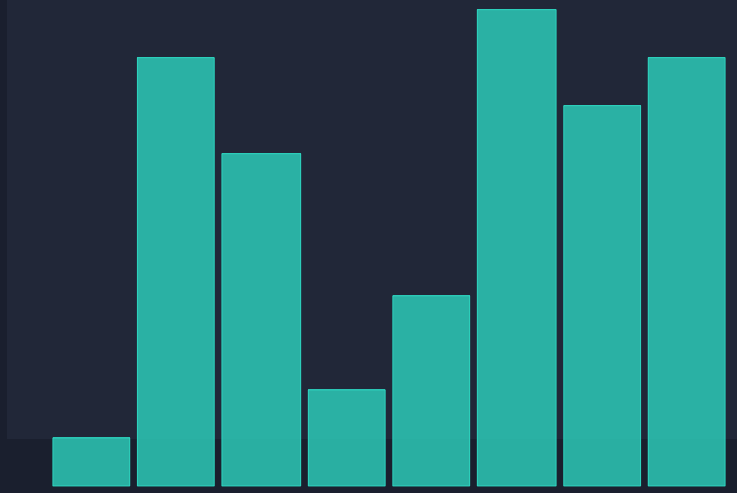
Three Samples, Three Stories (but often you only have ONE sample)

Each sample draws n=50 from the same trimodal population (mean=3.32, %>4=34%)

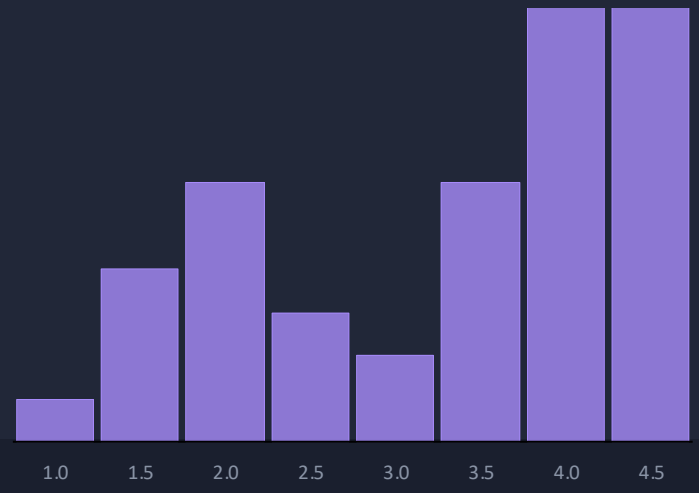
Sample A
Mean=2.89 Med=2.35
%>4=18%



Sample B Mean=3.31
Med=3.61 %>4=34%



Sample C Mean=3.74
Med=4.19 %>4=56%



Play at: https://reliableai.github.io – Reproduce in Notebook

Big assumption here (for now):

Our test samples are independent and have the same distribution of our production data

The only source of uncertainty – and of error in our estimate - is due to sampling

Why do we test with datasets that are not “large enough”?

- Getting test data is often hard
- Getting **GOOD** test data is **really hard**

Example: Think of analyzing customer support records by clustering them based on reported problems, inspecting steps to resolve the problem, and asking the LLM to aggregate steps per each cluster, so we can have solution workflows for each problem.

Now imagine having to generate good “ground truth” test data….

Approach

If possible, larger samples – but, alas, this is hard

The minimum we can do is report properly, via Beta Posteriors

E.g., Binarize scores (≥4) then fit Beta distribution

- Small n: wide credibility interval
- Large n: narrow posterior spike

Remember: Most people are scared when they hear “beta distribution”. But, they understand probability. So just point them to an online accuracy calculator inside the playground

What Even Is “One data point”?

Suppose your AI clusters customer support requests, then summarizes contents per cluster, then generate resolution workflows for problems discussed in that clusters.

What is “one data point” here?

Is it one record?

Is it one cluster?

Is it one generated workflow?

record → measures extraction; cluster → measures grouping quality; workflow → measures final utility.

Organizational awareness index

- Most org and most teams understand the notion of sample bias
- But, they forget that 100 data points may result in MASSIVE confidence intervals of nearly 20 points (try asking the question – even to seasoned data scientists)

02

Sampling Bias

The Problem: Wrong Population Mix

Test Set
(What You Measured)

95% English

5% Italian

Quality score: **4.2 / 5**

Production
(What Users Get)

60% English

40% Italian

Quality score: **3.2 / 5**

And this was the easy case...

Language was one obvious dimension. Bias lurks on many others:

Synthetic
vs Real

Simple
vs Complex

Formal
vs Casual

Generic
vs Domain

Clean
vs Noisy

A Common challenge with synthetic data generation

What You Think You Get

✓Unlimited scale on demand
✓Covers edge cases
✓Perfect label quality
✓Representative distribution

→

What You Actually Get

✗Oversimplified patterns
✗LLM style bias injected
✗Narrow distribution masking gaps

Synthetic data can confirm your model works on synthetic data
When you see synthetic data, ask or search for evidence it matches real data

How to solve

- No easy solution – also because we don't just need ONE test dataset, as we will see. we need many test sets
- Getting a small (50 or 100 or even 200) test set is fine for a spot check, not for iterating on it
- Ideally: use prod data and a judge model – you need ground truth at scale

If you really go with synthetic data…

01 **Map Your Distribution**

Know the real mix of languages, domains, and complexity levels your model sees in production

02 **Stratified Sampling**

Build your test set to mirror production proportions — not convenience samples

03 **Per-Segment Reporting**

Report scores for each segment separately. An overall mean hides failures on minority groups

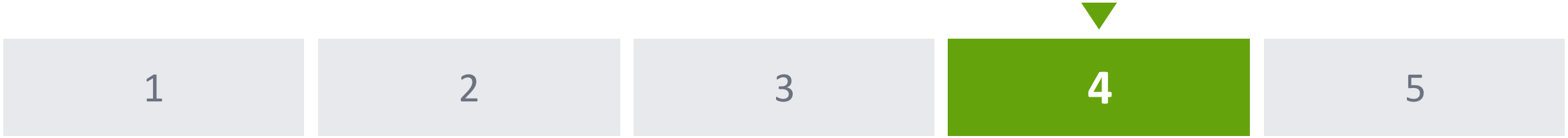
04 **Validate the “reliability” of synth data on Real Data**

Synthetic benchmarks are a starting point, not the finish line. Real queries expose real gaps

Does More Data Help?

NO – A bigger biased sample is still biased

Organizational awareness index



Low High

- Orgs are often unaware of how much synthetic data differ from real
- But are often aware there is variability across customers

How to improve: start asking the question.
e.g., ask your PM for a CI or uncertainty estimate around the quality number

03

Overfitting

The Problem: Information Leakage

How prompt and agent iteration creates a false sense of progress

Iterate on Dev

You tweak prompts using the same dev examples over and over

→

Encode Patterns

Prompt starts encoding specific patterns from dev data as few-shot examples

→

Dev Score Climbs

Dev accuracy rises with each iteration — you feel confident

→

Test Score Stalls

But held-out test performance plateaus or drops — gains were illusory

Your dev set is not trustworthy after many iterations

The Lab: Live Iteration

Judge Calibration

Prometheus + FeedbackBench
Dev: 50 | Test: 200 examples

Dev score improves with each iteration
Test score plateaus or declines

You tuned to the dev set, not the task

Structured Extraction

TWEETSUMM dataset
Tune prompt iteratively on dev set

Rules that work on dev don't generalize to test
Prompt becomes brittle and overspecific

You memorized quirks, not patterns

The Gap: Did we improve the model, or just the score on this set?

Does More Data Help?

Larger dataset delays overfitting, but won't prevent it

- Enough iterations will always find spurious patterns
- More data buys time, not safety

Value process and reporting discipline over data volume

Organizational Awareness Index

Danger zone:

- Engineers *know* overfitting exists, but underestimate how fast iterative prompt-tuning overfits a 200-example dev set. Management rarely asks.

04

Multiple Hypothesis Testing

You picked the luckiest variant

The Problem: Selection Bias

The Setup

- You design 10 prompt variants. Never peeked at test data
- Evaluate all 10 on same test set
- Variant #7 scores highest
- **You ship it**

- But: Variant #7 was just the luckiest
- On fresh data, it often performs worse than variants you discarded

Selection Bias Scales with N

N = 2 **±0.05** score inflation on a 1-5 scale

Small search — A/B test between two options

N = 10 **±0.15** score inflation on a 1-5 scale

Moderate search — Typical prompt optimization

N = 50 **±0.35** score inflation on a 1-5 scale

Exhaustive search — Grid search over many configs

More variants = bigger bias. The winner is always overstated.

The Fix: Corrections & Holdout /1

Two-Stage Evaluation

Stage 1: Dev set picks winner

Stage 2: Fresh test measures true score

Pre-Registration

Decide which variant to test before you look at results

The Fix: Corrections & Holdout / 2

Bonferroni Correction

Divide your significance threshold by N tests

$$\alpha' = \alpha / N$$

Keeps false positive rate honest

If you test N=10 variants at p<0.05, there's a ~40% chance one looks significant by luck.
Require p< **`0.05/10 = 0.005`** per variant to keep overall false-positive rate ≤5%.

Trade-off: conservative — may miss real winners. Holm-Bonferroni is a less strict alternative.

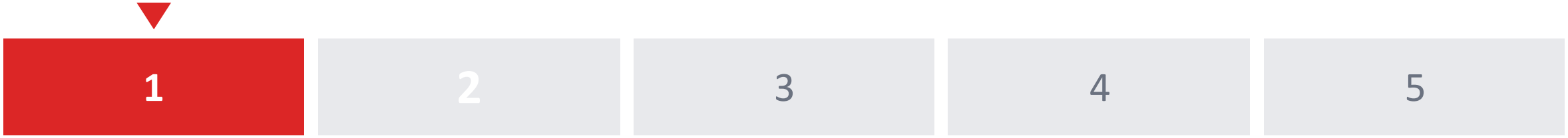
Does More Data Help?

PARTIALLY:
More data shrinks per-variant noise.
Selection bias gets smaller, never reaches zero.

Worse: tighter CIs make noise look 'significant',
so you're more confident in a biased pick.

Real fix: process (two-stage, corrections),
not just more data.

Organizational Awareness Index



Danger zone:

- Engineers *know* overfitting exists, but underestimate how fast iterative prompt-tuning overfits a 200-example dev set.
- The fix: Executives, managers and engineers need to start asking the question, raise awareness, amplify effort on real and realistic data collection at scale

05

Variance Across Domains

Different customers (or domains), different patterns

Overall Correctness: 4.0 (looks good)

Per-segment reality:

Finance queries: 4.6 → happy
Medical queries: 3.3 → churns
Legal queries: 3.8 → confused

What we can do

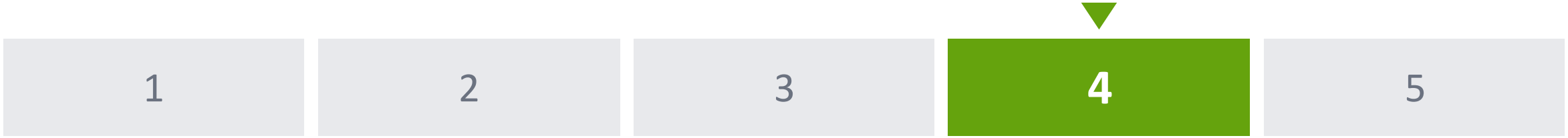
- Well… not much you can do to reduce data variability
- You can build ”better” (more adaptive) systems
- You can measure and take per customer or per domain decisions
- You can report in a way that makes this variability explicit

Does More Data Help?

NO More data gives tighter estimates.
Doesn't reduce the variance between segments.

Variance is a property of the world:
Your system handles finance well, medical poorly.

Organizational Awareness Index



Low High

Good (4/5)

06

Choice of Metric

The winner depends on how you ask

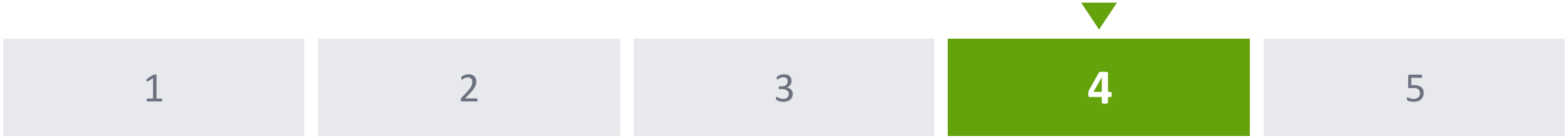
Same Data, Different Winners

System A vs B — 200 queries, same judge

Mean: A 4.1 vs B 3.9 → A wins
Median: A 4.0 vs B 4.0 → Tie
% scoring ≥4: A 62% vs B 68% → B wins
5th percentile: A 1.8 vs B 2.5 → B wins

A has higher average. B is more consistent.
Which is 'better'? The data didn't change. The question changed.

Org awareness



Good 4/5:

07

Temporal Drift

Your number has an expiration date

The Problem: The World Changes

Customer support bot — monthly evaluations:

January: Correctness 4.2 → ship it
February: Correctness 4.1
March: Correctness 3.8 → hmm
April: Correctness 3.5 → panic

Code never changed. World changed.
New product launch → queries the system was never designed for.
Your January evaluation tells you about January. Nothing about April.

The Fix: Continuous Monitoring

1. Build monitoring into the system from day one.

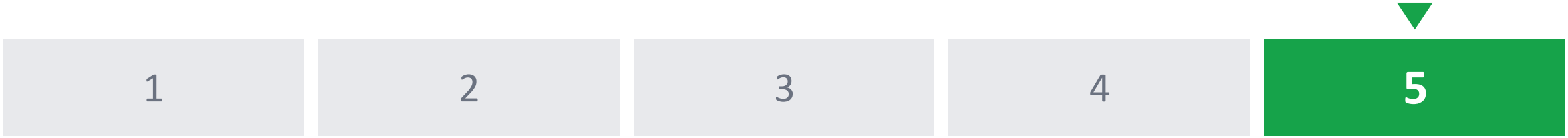
2. Automated periodic evaluation on fresh production data.

3. Drift detection: alert when scores drop or shift too much.

4. Version everything (data, prompts, models)
   to diagnose what drifted.

Does more data help? NO — more old January data
tells you about January. Need fresh data over time.

Org awareness



Low High

Very good 5/5

08

Prompt Brittleness

One comma changed your accuracy by 20%

Real Research Findings

Reorder few-shot examples: 54% → 93% accuracy [1]
Change single delimiter char: ±23% accuracy [2]
Formatting in few-shot: Up to 76 accuracy points [2]
Move answer option positions: Model ranking changes [3]

Sclar et al. deep dive: Same task, same model, same examples — only whitespace differs.

Format A: `"Input: X\nOutput: Y"` → **85% acc**

Format B: `"Input: X  Output: Y"` → **9% acc**

~10pt avg spread across 50+ tasks, all models. These aren't corner cases.

[1] Zhao et al. "Calibrate Before Use" ICML 2021 [2] Sclar et al. "Quantifying LMs' Sensitivity" ICLR 2024

[3] Zheng et al. "LLMs Are Not Robust Multiple Choice Selectors" ICLR 2024

The Fix: Sensitivity Testing

Before shipping, try N rephrasings.
Measure variance in scores.

Report score as a range, not a single number.
Best / worst across rephrasings.

Robustness as a selection criterion:
Prefer prompts where small changes → small score changes.

Pin prompt versions exactly.
Treat any edit as a new deployment.

Does More Data Help?

NO: More data makes each prompt variant
more precise, but doesn't reduce brittleness.

The sensitivity between prompts is a property of the model,
not the measurement.

You can measure it more precisely with more data.
The gap between variants stays.

Many other sources of brittleness: eg Model

The provider updated the weights silently.
Your prompt, test set, code: unchanged.
Scores shifted.

Key example — Chen et al. (2023):
GPT-4 on 'identify prime numbers'
March 2023: 97.6% accuracy
June 2023:   2.4% accuracy

Catastrophic regression. Same task, same prompt.
22.9% of model updates cause prompt regressions (>20% drops).

The Judge Layer

Every source applies again

Every Measurement Goes Through the Judge

The judge (LLM or human) is itself an imperfect system.
Every uncertainty source applies AGAIN at the judge level.

Judge issues:

- **Noise**: same judge, same output, different score
- **Bias**: systematically generous or harsh
- **Subjectivity**: experts disagree on 'what is a 4?'
- **Brittleness**: judge prompt rewording → different scores
- **Evolution**: judge model gets updated

Judge uncertainties MULTIPLY with others. They don't add.

The Compounding Effect

Perfect judge scenario:
Sampling noise alone: CI ±0.2

Add judge noise (65% self-agreement):
Each data point is itself uncertain: CI ±0.4

Add judge bias (systematically +0.3):
CI still ±0.4, but center shifts to 4.4

Add judge subjectivity:
There isn't even a single 'true' value to estimate.

Noisy judge doesn't just add fuzz — it doubles/triples uncertainty.

Subjectivity

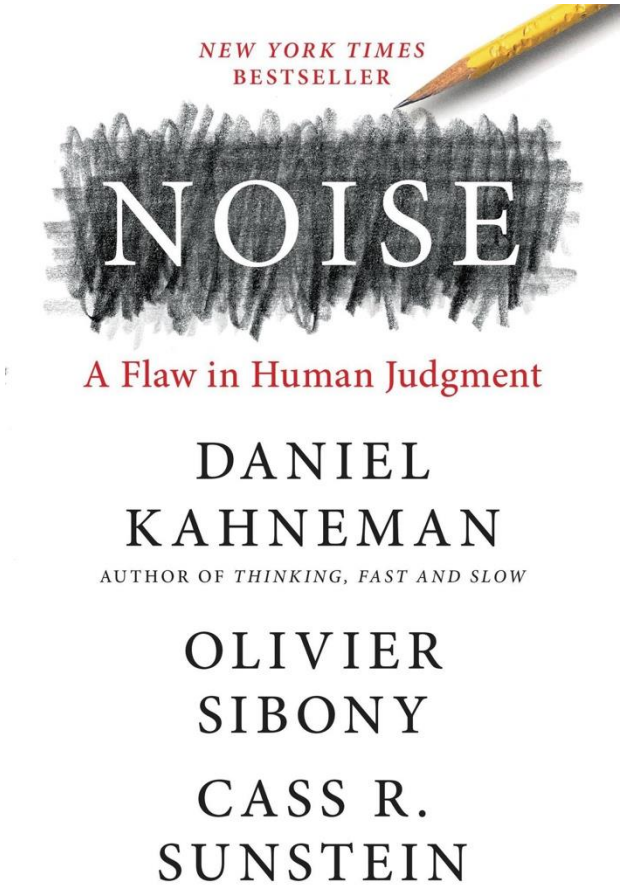
Some things are genuinely subjective.
"Is this response helpful?" Reasonable experts disagree.
There isn't a ground truth to converge to.

Example: 20 outputs, 5 human experts score Helpfulness
Some outputs: all experts agree (4-5)
Others: range from 2 to 5

If experts disagree, what is the 'right' score?
Your ground truth is a social construction, not a fact.
The LLM judge adopted one subjective standard.

Watch the Uncertainty Grow

What's included	Score	Uncertainty
Perfect judge, infinite data	4.10	Exact
+ Sampling noise (n=100)	4.10 ± 0.15	CI narrows
+ Judge noise (65% agreement)	4.10 ± 0.35	Each point uncertain
+ Judge bias (systematically +0.3)	4.40 ± 0.35	Shifted center
+ Sampling bias (wrong population)	4.40 ± 0.35	Truth outside CI
+ Prompt brittleness (±0.3)	3.4–4.8	Multiple values
+ Model evolution (judge updated)	???	Instrument changed



"Most organizations prefer consensus and harmony over dissent and conflict."

Organizations rarely recognize and accept the existence of noise (bias, too)

My buddy Andy: "yeah, there is a standard deviation"

Visibility, Culture, Action

1. The *visibility* problem (epistemic): Are we aware of the uncertainty?

2. The *culture* problem (organizational): Do we ask about it? Do we ignore it? Do we reward quantifying it?

3. The *action* problem (methodological): What can we do about it?

Interactive Tools & Playgrounds

Explore the sources interactively:

- sampling_noise.html — adjust n, watch CI collapse
- sampling_bias.html — mix biased samples, see the gap
- multiple_hypothesis_testing.html — N variants, N winners
- variance_across_domains.html — Simpson's paradox live
- choice_of_metric.html — same data, different winners
- temporal_drift.html — monthly scores over time
- stacking.html — watch uncertainty compound

Go play with it. Get a feel for how these sources hide in your measurements.

Eval in Software 1.0 vs 2.0 vs 3.0

How evaluation paradigms evolve across software generations

	Software 1.0	Software 2.0	Software 3.0
Nature of use cases	Features are clear and well-described. Output is measurable and predictable.	Relatively small number of distinct use cases in production. Model predictions on structured data.	Complex, often subjective use cases. Many things we want from a "good" answer. Eval is multifaceted.
Ground truth availability	Reviewed and approved test cases. Clear expected behavior.	Relatively easy to get ground truth data. Large test sets available.	Hard to get ground truth – esp. if it requires human labeling. Hard to even know if an answer matches the ground truth.
Metrics & measurement	Output is measurable. Regression and updates checked against test cases.	Metrics perceived to be understood. Gap between what is measured in literature and what matters in business. Harder than people think.	Subjectivity and disagreement among labelers must be measured. Hard to define what "correct" means. Eval is multifaceted – many dimensions of quality.
Who leads eval	PMs and QA engineers.	PMs and data scientists.	Wide range of people and roles (SW eng, PMs, process owners). Unclear RACI.
Approach to scale	Test cases + acceptance testing. Deterministic: run once, trust the result.	Large datasets, including large test sets. Scale through data volume.	LLM as a judge as a potentially useful approach to get scale. Still requires calibration.